Server Sync

A multithreaded e-commerce web server written from scratch in C++17 with MySQL
PBL Evaluation Guide · Graphic Era

Team and roles

Member	Role	Main files
Sneha	Database	`database/*.sql`, `backend/db/*`, product, category, cart, order and admin services
Shoyam	Threads	`main.cpp`, `config.*`, `core/server.*`, `core/log.*`, admin logs page, build/run scripts, README
Ashutosh	HTTP and sessions	`core/http.*`, `app.*`, `user_service.*`, auth and shop handlers, `view.*`, storefront templates, `style.css`, `app.js`
Aradhana	Sockets and routing	`core/router.*`, `routes.*`, `core/template.*`, cart, order and admin handlers, admin templates, `admin.css`

1. Problem statement

A website like an online store is used by **many people at the same time**. If the server handles only one request at a time, everyone else has to wait. And when two people buy the **last item in stock** at the same moment, a careless server can sell it twice or leave the database half-updated.

Most students build websites with ready-made frameworks (Node.js, Django), which hide these problems. We wanted to build the server ourselves so we could **see and solve these problems directly**.

2. Objectives

1. Build a working **web server from scratch in C++** using TCP sockets, with no web framework.
2. Serve **many users at once** using a **thread pool** and protect shared data with **mutexes**.
3. Store all data in a **normalised MySQL database** with primary and foreign keys.
4. Make checkout **safe** using **transactions** and row locks, so stock is never oversold.
5. Add **login and sessions** with hashed passwords and cookies.
6. Provide a complete **storefront** for customers and an **admin panel** for managing the store.

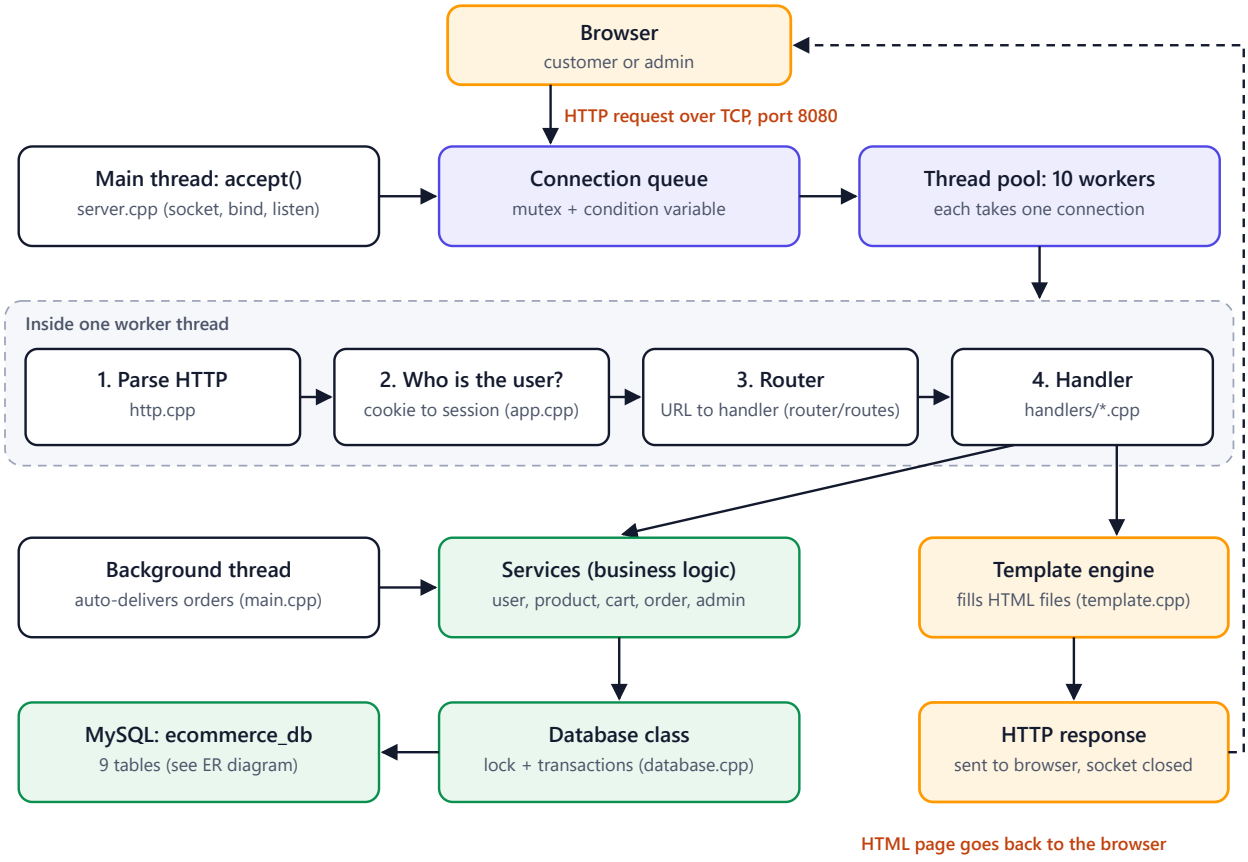
3. Proposed solution (in one paragraph)

Server Sync is an online store. Customers browse products, add them to a cart, check out (cash on delivery) and track or cancel orders. Admins manage products, categories, orders, users and sessions, and watch a **live log** showing which worker thread handled each request and which SQL it ran. Everything from opening the socket and reading HTTP to filling the HTML pages is our own C++ code. Only MySQL is an external system.

> **Tech stack:** C++17 · Winsock2 (TCP sockets) · `std::thread`, `std::mutex`, `std::condition_variable` · MySQL + MySQL C API · HTML, CSS, a little JavaScript · Windows (`build.bat`).

4. System architecture

The diagram shows the path of **one request** through the system. Each box names the source file responsible.



The layers in plain words

Layer	What it does
Network + threads	Opens a TCP socket on port 8080. The main thread only *accepts* connections and puts them in a queue. 10 worker threads take connections from the queue and serve them.
HTTP + sessions	Turns raw bytes like `GET /cart HTTP/1.1 ...` into a `Request` object, reads the `session_id` cookie and finds the logged-in user.
Routing	A table of about 37 URLs. For example, `POST /cart/add` runs `cartAdd`. It also checks access rules ("must be logged in", "must be admin").
Handlers + services	Handlers read the form and choose the page. Services hold the rules (stock checks, totals) and run the SQL.
Database	One shared MySQL connection protected by a lock. Checkout runs inside a **transaction**.
Templates + frontend	The C++ code never writes HTML directly. It fills placeholders like `{{name}}` in HTML files under `frontend/templates/`.

5. How it works: one click, step by step

Example: a logged-in customer clicks **"Place order"**.

1. **Browser** sends `POST /checkout/place` over TCP with the cookie `session_id=…`.
2. **Main thread** runs `accept()`, pushes the socket into the queue and wakes one sleeping worker (`notify_one`).
3. **Worker 3** (for example) takes the socket and reads the full request (headers + body, with size limits).
4. **http.cpp** parses the method, path, headers, cookies and form fields.

5. **app.cpp** looks up the session in `user_sessions`, so the server now knows this is a particular user.
6. **Router** matches `POST /checkout/place`, checks *requireLogin* and calls `placeOrderSubmit`.
7. **order_service** runs one **transaction**: `START TRANSACTION` → lock cart rows (`SELECT … FOR UPDATE`) → check stock → insert `orders` + `order_items` → reduce stock (only `WHERE stock >= qty`) → insert `payments` (COD, PENDING) → mark cart `CHECKED_OUT` → `COMMIT`. If **any** step fails, everything is **rolled back**.
8. The handler sends a **redirect** to *My orders*, and the **template engine** builds the page.
9. The worker sends the response, closes the socket, **records the request in the admin log** (worker id, time taken, SQL run) and waits for the next connection.
10. A **background thread** checks every 5 seconds and marks orders `DELIVERED` 1–2 minutes after checkout. Until then the customer can cancel, which puts the stock back.

The two main concurrency problems and how we solved them

Problem	Our solution
Many threads use the same queue at once	**Producer–consumer pattern:** the queue is protected by a `mutex`. Idle workers sleep on a `condition_variable` instead of wasting CPU.
Two buyers want the last item at the same moment	**Transaction + row lock + safe UPDATE:** the second buyer sees "not enough stock" and gets a full rollback, so stock never goes below zero.
Many threads share one MySQL connection	A `recursive_mutex` in the `Database` class lets only one thread talk to MySQL at a time.
Category list is cached in memory	The cache has its own small `mutex`.

Main features (current implementation)

Customer storefront

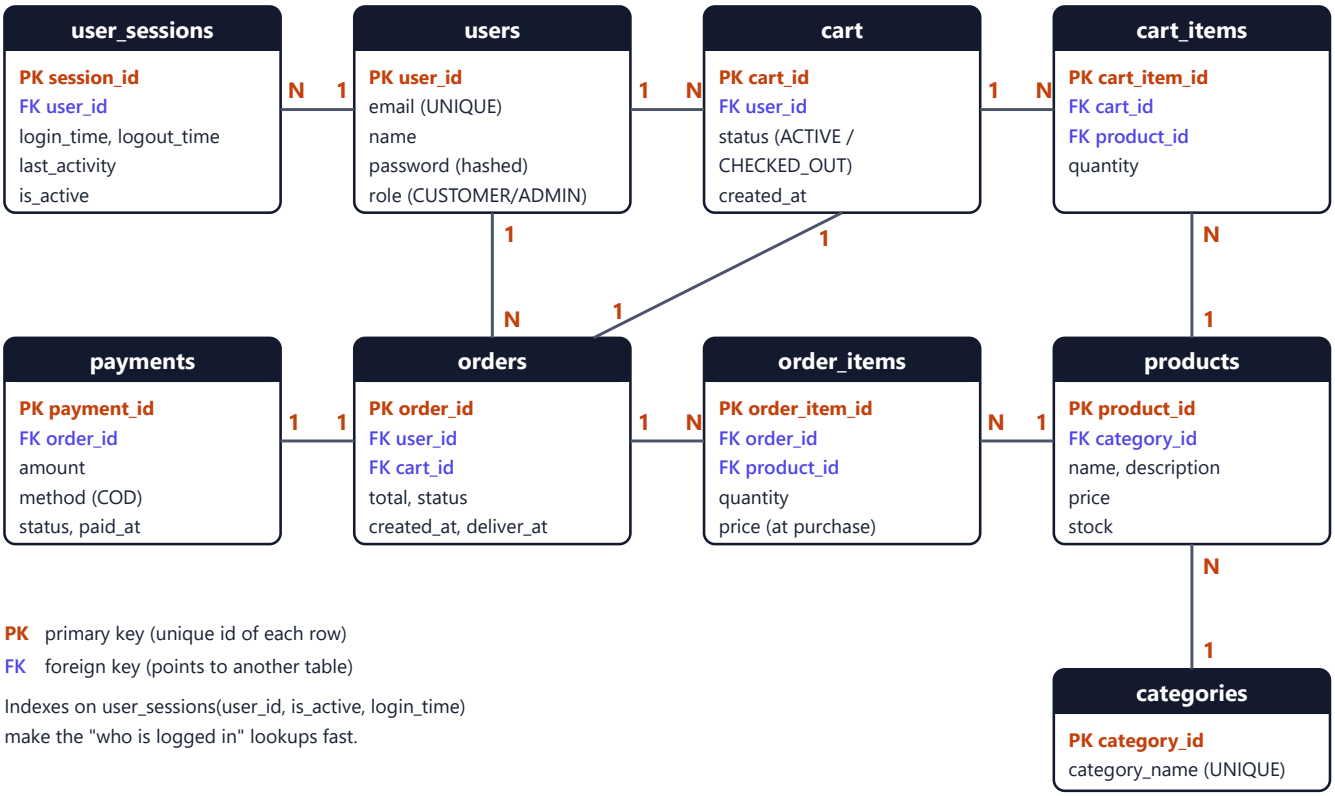
- Sign up, log in, log out (salted SHA-256 passwords)
- Home page; product list with search, category, price filter and sort; product detail
- Cart: add, change quantity, remove
- Checkout (cash on delivery), my orders, cancel order
- See your own active login sessions

Admin panel

- Dashboard with key numbers and low-stock alerts
- Add, edit and delete products; manage categories
- Orders: change order and payment status
- Users: change role; sessions: force logout
- Database table viewer; **live request log** (worker thread, time, SQL)

6. Database design: ER diagram

9 tables in the MySQL database `ecommerce_db`. **PK** = primary key, **FK** = foreign key. A **1** at one end and an **N** at the other means one-to-many.



Relationships in words

- A **user** can have many **sessions** (one per browser login), many **carts** over time and many **orders**.
- A **cart** has many **cart_items**. When it is checked out, it becomes exactly **one order**.
- An **order** has many **order_items** and one **payment**.
- A **category** has many **products**. A product can appear in many cart_items and order_items.

Design decisions to mention in the viva:

- `order_items.price` stores the price *at the time of purchase*, so later price changes do not alter old bills.
- `cart_items` and `order_items` are **bridge tables** that resolve the many-to-many link between carts/orders and products.
- `UNIQUE` on email and category name, and `NOT NULL` plus foreign keys, keep the data consistent.

7. Integration of PBL subjects

Check before the evaluation: the subject names below are inferred from the code and the team roles. Replace them with the exact PBL subject names from your syllabus.

Subject	Concept used	Where in our project

Operating Systems	Multithreading, thread pool	10 worker threads in `server.cpp`
	Producer–consumer, condition variable	Accept loop (producer) → queue → workers (consumers)
	Mutual exclusion, critical section, race condition	Mutex on queue, database, category cache, log buffer
	Background (daemon) thread	Auto-delivery thread in `main.cpp`
DBMS	ER modelling, normalisation, PK/FK, constraints	`ecommerce_db.sql`, 9 tables
	Transactions and ACID, COMMIT/ROLLBACK	`Transaction` class, `placeOrder`, cancel order
	Concurrency control (row locks)	`SELECT … FOR UPDATE`, `UPDATE … WHERE stock >= qty`
	Joins, aggregates, indexes, SQL injection safety	Services and the admin dashboard; `db.quote()` escapes user text
Computer Networks	TCP sockets: socket, bind, listen, accept, recv, send	`server.cpp` (Winsock2)
	HTTP protocol: methods, headers, status codes	`http.cpp`: GET/POST, 200/302/404/500
	Cookies and state over a stateless protocol	`session_id` cookie (HttpOnly, SameSite=Lax)
OOP / C++	Classes, RAII, STL, separation of layers	`Database`, `Transaction`, `Router`, `Request/Response`

8. Why our approach (and its honest limits)

Why a thread pool, not "one thread per user"?

Creating a new thread for every request is slow and can exhaust memory under load. A fixed pool of 10 threads is created once and reused, so the load stays controlled.

Why no framework?

Frameworks hide sockets, parsing and threading. Building them ourselves is exactly what lets us show OS, CN and DBMS concepts working together.

Known limitations

- One shared MySQL connection behind a lock means database work is done **one thread at a time**. A **connection pool** would fix this.
- Plain HTTP only (no HTTPS) and Windows-only (Winsock).
- Cash on delivery only. No real payment gateway.
- No automated load test numbers yet (see milestones).

> **Tip:** evaluators like it when you name a limitation before they find it. Say "this is a known limitation and here is how we would fix it."

9. Individual contributions (about 5 minutes each)

Each person: 1 min on the problem and architecture (pages 1–2), 3 min on your own part (below), 1 min on progress and next steps. Open your own files on screen while you explain.

Sneha — Database

Built: the database schema and seed data (`ecommerce_db.sql`), the `Database` class (connect, run queries, escape text, `Transaction` with automatic rollback), and the product, category, cart, order and admin services.

Explain: the ER diagram, PK/FK, why `order_items` stores price, and the checkout transaction (step 7 on page 3).

Likely questions: What is ACID? · What does `FOR UPDATE` do? · Which normal form are the tables in? · How do you stop SQL injection?

Shoyam — Threads

Built: program start-up (`main.cpp`), settings (`config.*`), the socket server with the **thread pool and queue** (`server.cpp`), the background delivery thread, the request logger (`log.*`) and the admin **live logs** page, plus `build.bat`, `run.bat` and the README.

Explain: producer–consumer with `mutex` + `condition_variable`, and show the live log page where different worker IDs handle requests.

Likely questions: What is a race condition? · Why use a condition variable instead of a loop? · What is a deadlock and could it happen here? · Why 10 threads?

Ashutosh — HTTP and sessions

Built: the **HTTP parser and response builder** (`http.cpp`: request line, headers, URL decoding, form data, cookies), request handling in `app.cpp` (static files, finding the user from the cookie), the **user service** (sign up, log in with salted SHA-256, random session IDs, logout), the auth and shop handlers (home, product list with filters, product detail), view helpers, the storefront templates, `style.css` and `app.js`.

Explain: show a raw HTTP request and how it becomes a `Request` object; how login creates a row in `user_sessions` and sets an **HttpOnly cookie**; how every later request finds the user from that cookie.

Likely questions: HTTP is stateless, so how do you remember the user? · GET vs POST? · Why hash and salt passwords? · What do HttpOnly and SameSite do? · Why redirect after a POST?

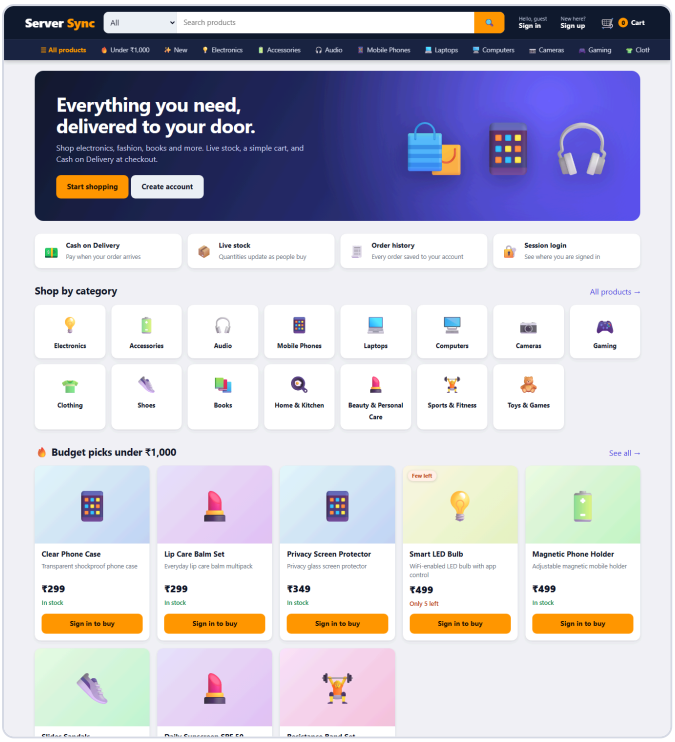
Aradhana — Sockets and routing

Built: the **router** (map from "METHOD path" to handler, with `requireLogin` / `requireAdmin` guards), the **site map** in `routes.cpp`, the **template engine** (`{{x}}`, `{{#if}}`, `{{#each}}`, partials, HTML escaping), the cart and order handlers, all admin handlers and admin templates, and `admin.css`.

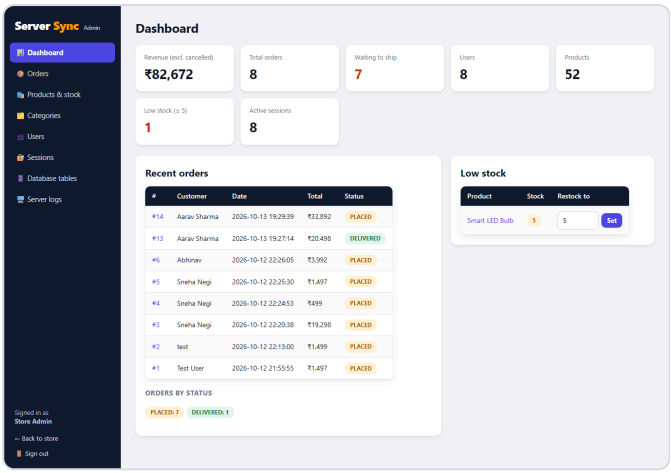
Explain: how a request that arrives on the socket is matched to a handler, how a customer is blocked from `/admin`, and how a template is filled. Be ready to explain socket, bind, listen, accept and send.

Likely questions: What happens on a URL that does not exist (404)? · Why escape HTML (XSS)? · What is a TCP socket and port? · How does the server send a response?

10. Current progress



Storefront: home page



Admin panel: dashboard

Part	Status
Socket server + thread pool + request queue	Done
HTTP parsing, cookies, sessions, password hashing	Done
Router (about 37 URLs) + template engine	Done
Database schema, seed data, transactions	Done
Storefront: browse, cart, checkout, orders, cancel	Done
Admin: products, categories, orders, users, sessions, tables, live logs	Done
Load testing with measured results	Planned

11. Planned milestones and expected outcomes

> **Edit to match your team's real plan.** These are suggestions based on the current limitations.

#	Milestone	Expected outcome
1	Load test (e.g. 100 parallel users with a script or a tool like ApacheBench)	Numbers (requests/sec, average time) comparing 1 thread with 10 threads
2	Concurrency test: many users buying the last item at once	Proof that stock never goes below 0 and no order is half-saved

3	MySQL connection pool	Database work runs in parallel, giving faster responses under load
4	Security hardening: HTTPS, CSRF tokens, login rate limit	A safer login and checkout
5	Final report, demo video and viva preparation	A complete PBL submission

> **How we will measure success:** all features working in the demo, zero overselling in the concurrency test, faster responses with the thread pool than with a single thread, and every member able to explain any part of the system.

12. Quick viva answers (everyone should know these)

What is our project in one line?
An online store whose web server we built ourselves in C++. It serves many users at once using threads and keeps orders correct using database transactions.

What is a thread? Why use multithreading?
A thread is a separate path of execution inside one program. With 10 workers, 10 requests are handled at once, so one slow request does not block others.

What is a race condition? How did we prevent it?
Two threads change shared data at the same time and the result depends on timing. We protect shared data (queue, DB connection, cache, log) with a `mutex`, so only one thread enters at a time.

What does the condition variable do?
Idle workers sleep until the main thread puts a connection in the queue and wakes one with `notify_one()`. They do not waste CPU checking again and again.

What is a transaction? ACID?
A group of SQL statements that succeed or fail together. **A**tomic (all or nothing), **C**onsistent (rules hold), **I**solated (others do not see half-work), **D**urable (saved after commit).

Two people buy the last item at the same time. What happens?
The first transaction locks the rows. The second waits, then sees stock is 0, shows "not enough stock" and rolls back. Stock never becomes negative.

HTTP is stateless. How does the site remember you?
At login we create a random `session_id`, save it in `user_sessions` and send it as a cookie. The browser sends it back with every request, and we look it up.

How are passwords stored?
Never as plain text. They are stored as a **salted SHA-256 hash** (`sha256$salt$hash`), so even a database leak does not reveal them.

How do you stop SQL injection and XSS?
SQL injection: all user text is escaped with `db.quote()` before it goes into SQL. XSS: `{{value}}` in templates is HTML-escaped.

What is TCP vs HTTP?
TCP is the reliable connection that carries the bytes (the socket). HTTP is the text format of the messages sent over it (`GET /cart HTTP/1.1` ...).

What would you improve?

Add a database connection pool, HTTPS, measured load tests, and online payments.

> **Before the evaluation:** run `build.bat` and `run.bat` once and keep the site open. Have the **admin live log page** ready, because it shows different worker threads handling requests, which is the best live proof of multithreading.